

Analysis Components & Techniques

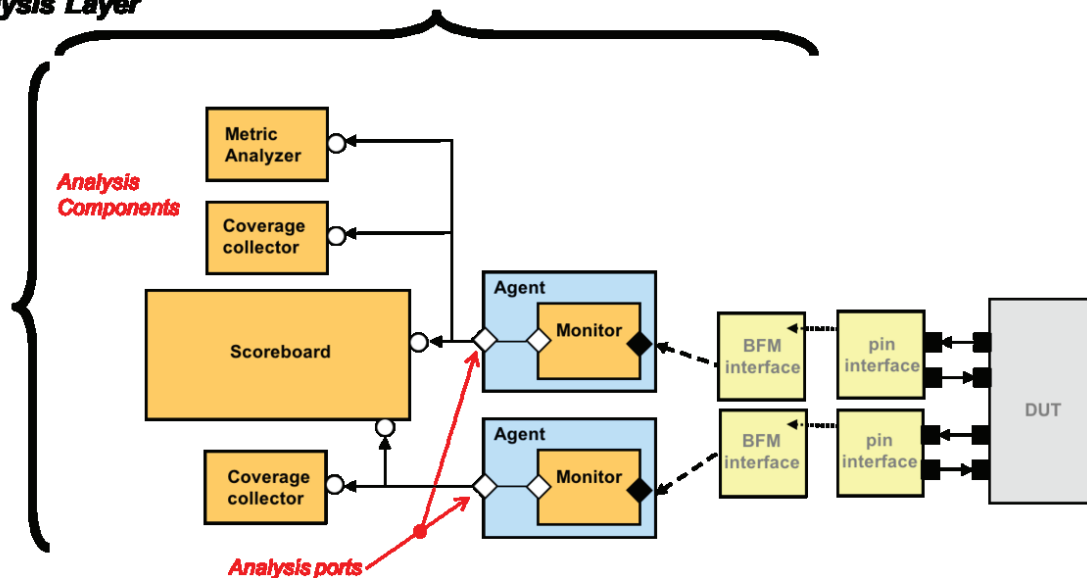
Analysis Components and Techniques

Topic Overview

Simply put, design verification involves two primary aspects: (i) the generation of input stimulus to put the design in a desirable, known state, and (ii) the analysis of corresponding results in the form of design outputs in response to the stimulus.

The analysis portion of a testbench is made up of components that observe behavior and make a judgement whether or not it is as expected in accordance with a design specification, thus providing confidence that the design conforms to its specification. Examples of specified behavior include functional behavior, performance, as well as power utilization.

Analysis Layer



Monitoring DUT Activity

The analysis part starts its decision process by observing the DUT responses. This is done by one or more monitors (proxy and BFM) that observe the signal-level activity on the DUT interfaces (via virtual interfaces). A UVM monitor converts signal-level activity into TLM transaction objects and broadcasts these to interested analysis components - i.e. subscribers - via analysis port connections. These subscribers capture the transactions and conduct their analysis.

Scoreboards

Scoreboards are analysis components that collect the transactions sent by a monitor and perform specific (often comparative) analysis computations to determine whether or not the design is functioning as expected. A good scoreboard architecture separates its tasks into two areas of concern: prediction and evaluation.

A predictor model, sometimes referred to as "Golden Reference Model", receives the same stimulus transactions as the DUT and computes known good response transactions. The scoreboard evaluates the predicted with respect to

the actual responses. The most common evaluation technique uses a comparator of some sort, capable of comparing transaction streams in in-order or a particular out-of-order fashion.

Coverage Collectors

Coverage information is paramount to answer the questions "Are we done (testing) yet?" or "Have we done enough (testing) yet?"

Coverage collectors are analysis port subscribers that sample observed transactions and activity into SystemVerilog functional coverage groups. The coverage data collected from each test is stored into a shared coverage database used to determine overall verification progress.

Metric Analyzers

Metric Analyzers watch and record non-functional behavior such as timing/performance and power utilization. They generally exhibit a standard architecture. Depending on the number of observed transaction streams, they are implemented as a `uvm_subscriber` or with analysis exports. Metric analyzers perform ongoing calculations during the run phase, and/or during the post-run phases.

Analysis Reporting

All data for analysis is collected during simulation. Most analysis takes place dynamically during the course of the simulation run (i.e. on-the-fly), though some analysis may be performed after the UVM `run_phase` ends when timed simulation completes. UVM provides three phases for this purpose: `extract_phase`, `check_phase`, and `report_phase`. These post-run phases provide components with the option to extract relevant data collected on-the-fly during the run, perform a post-run check, and finally produce comprehensive reports on all analysis results.

Analysis Port

Overview

One of the unique aspects of the analysis section of a testbench is that usually there are many independent calculations and evaluations all operating on the same piece of data. Rather than lump all these evaluations into a single place, it is better to separate them into independent, concurrent components. This leads to a topological pattern for connecting components that is common to the analysis section: the one-to-many topology, where one connection point broadcasts information to many connected components that read and analyze the data.

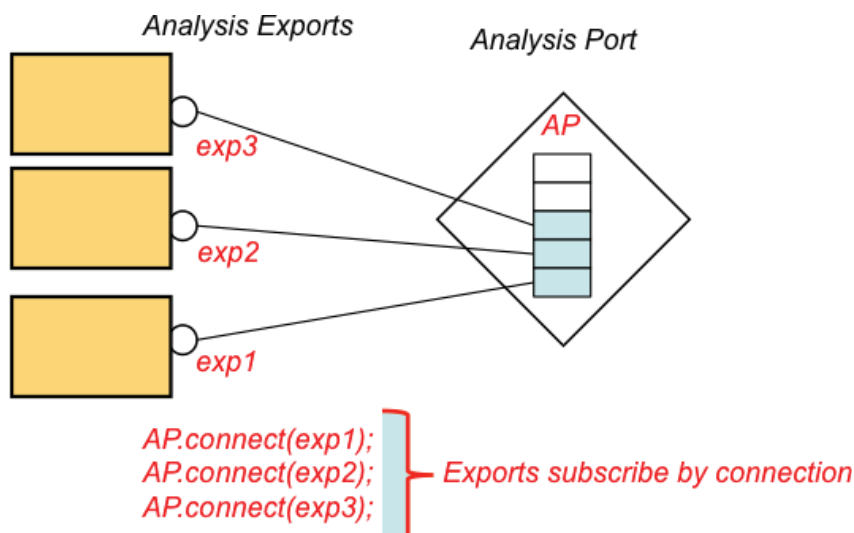
This connection topology implementation behavior lends itself to an OOP design pattern called the "observer pattern". In this pattern, interested "observers" register themselves with a single information source. There is no minimum or maximum number of observers (e.g. the number of observers could be zero). The information source performs a single operation to broadcast the data to all registered observers.

An additional requirement of UVM Analysis is "Do not interfere with the DUT". This means that the act of broadcasting must be a non-blocking operation.

UVM provides three objects to meet the requirements of the observer pattern as well as the requirement of non-interference: analysis ports, analysis exports, and analysis fifos.

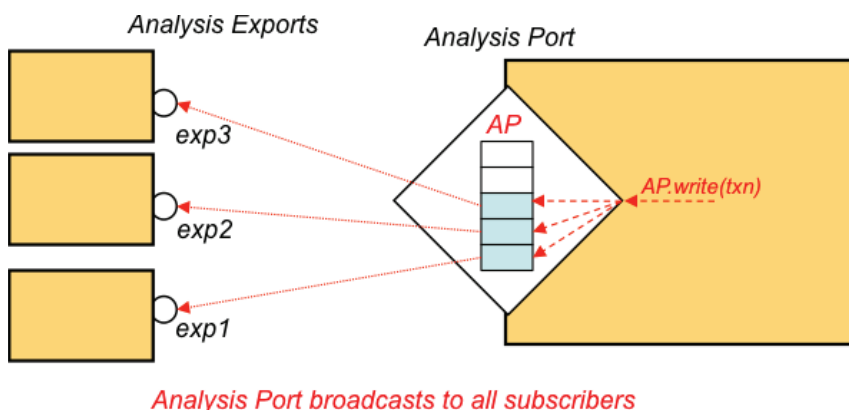
Detail

Analysis ports, analysis exports, and analysis fifos follow the standard UVM transaction-level communication semantics. An analysis port **requires** an implementation of `write()` to be connected to it. An analysis export **provides** the implementation of the `write()` function. As with other UVM TLM ports, analysis ports are parameterized classes where the parameter is the transaction type being passed. Ports provide a local object through which code can call a function. Exports are connection points on components that provide the implementation of the functions called through the ports. Ports and exports are connected through a call to the `connect()` function.



All other UVM TLM ports and exports, such as blocking put ports and blocking put exports, perform point-to-point communication. Because of the one-to-many requirement for analysis ports and exports, an analysis port allows multiple analysis exports to be connected to it. It also allows for no connections to be present. The port maintains a list of connected exports.

The Analysis port provides a single void function named `write()` to perform the broadcast behavior. When code calls the `write()` function on an analysis port, the port then uses its internal list to broadcast by calling `write()` on all connected exports. This causes the `write()` function to be executed on all components containing the connected exports. If no exports are connected to the analysis port, then no action is performed when the `write()` function is called and no error occurs.

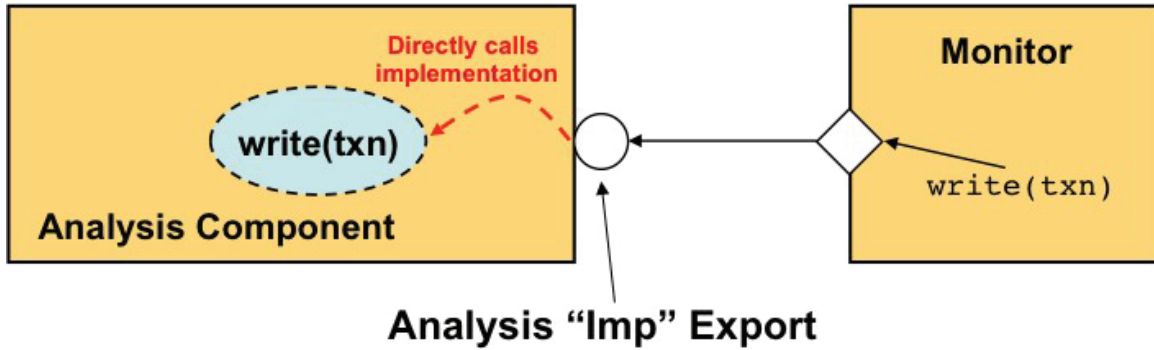


Analysis ports adhere to the non-interference requirement by providing the `write()` operation as a function, rather than a task. Because it is a function, it cannot block.

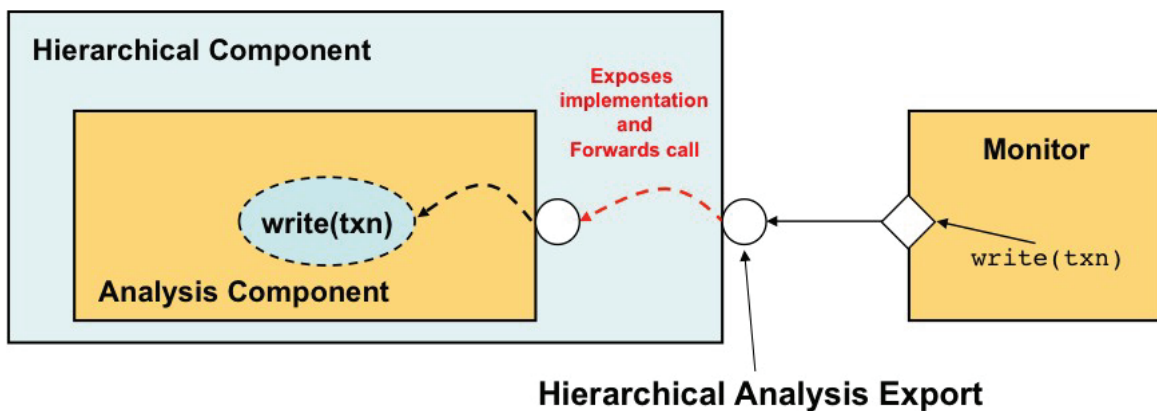
Analysis Exports

You register an analysis export as an observer with an analysis port by passing the export as the argument to the analysis port's `connect()` function. As with other TLM exports, an analysis export comes in two types: a hierarchical export or an "imp" export. Both hierarchical and "imp" exports can be connected to a port.

An "imp" export is placed on a component that actually implements the `write()` function directly.

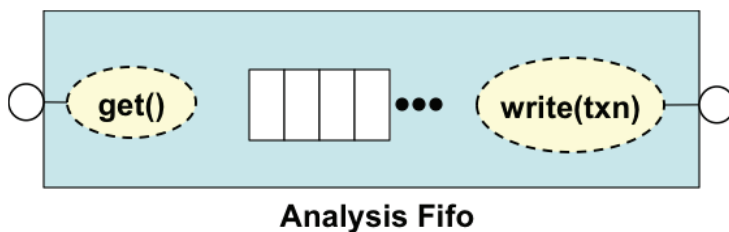


A hierarchical export is used for hierarchical connections, where the component that implements the `write()` function is a hierarchical child of the component with the export. A hierarchical export forwards the call to `write()` to the child component.

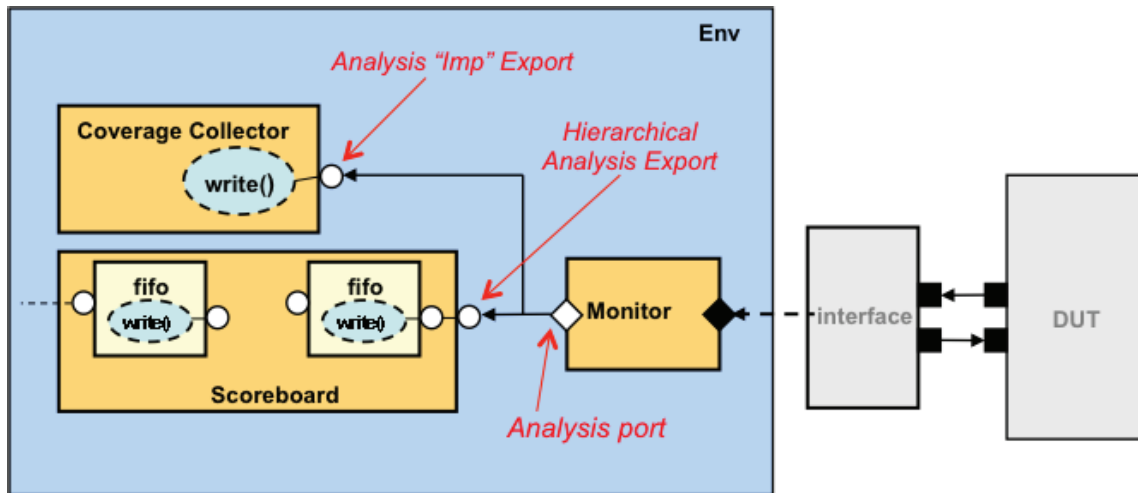


Analysis Fifos

UVM provides a pre-built component called an analysis fifo that has an "imp"-style analysis export and an implementation of the `write()` function that places the data written into a fifo buffer. The buffer can grow indefinitely, in order to prevent blocking and adhere to the non-interference requirement. The analysis fifo extends the `tlm_fifo` class, and so it also has all of the exports and associated operations of a `tlm_fifo` such as blocking `get`, etc.



Implementing the write() function



The analysis component is required to implement the `write()` function that is called by the Monitor's analysis port. For an analysis component with a single input stream, one can extend the `uvm_subscriber` class. For components that have multiple input streams, one can either directly implement "write()" functions and provide "imp" exports, or one can expose `write()` function implementations of hierarchical children by providing hierarchical exports. The preferred choice is generally dependent on the intended functionality of a component as well as one's preferred coding style. In the diagram above, the Coverage Collector extends the `uvm_subscriber` class which has an analysis "imp" export. The extension then implements the `write()` function.

The Scoreboard has two input streams of transactions and uses embedded Analysis fifos to buffer incoming transactions. In this case the `write()` method is implemented in the fifos, hence the Scoreboard has hierarchical analysis exports to expose the `write()` method externally.

Analysis Connections

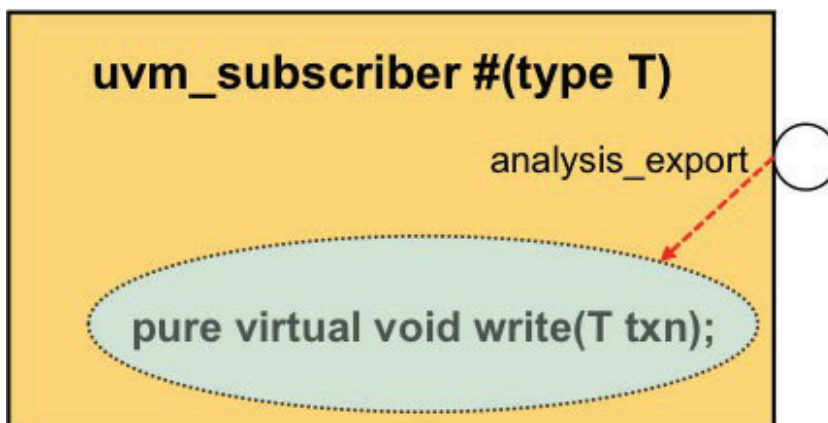
Overview

An analysis component such as a Monitor sends transactions to another analysis component through a TLM connection which is a chain of objects where each calls the `write(t)` function in the next. The monitor or other analysis component starts the chain by calling a `write(t)` function in the analysis port object, which calls `write(t)` in the next analysis object(s), until the final object calls `write(t)` in the component. The final TLM object is called an analysis imp as it calls the component that has the implementation of the `write(t)` function.

The chain of calls starts with an analysis port (`uvm_analysis_port`) which may be connected to another `analysis_port` to move up in the testbench hierarchy (Eg. monitor level to agent level). An analysis port may also be connected to an analysis export (`uvm_analysis_export`). This connection is made when at the same level of a testbench hierarchy (Eg. in the environment). When moving back down the hierarchy, the analysis export can be connected to another export or an analysis imp (`uvm_analysis_imp`). The imp ends the chain of calls.

Single Transaction Stream: `uvm_subscriber`

Many analysis components just need to sample the data sent by the Monitor, and perform some calculation such as coverage measurement. For these components, which only deal with a single stream of transactions, UVM provides a base component class called `uvm_subscriber`. This class extends `uvm_component` and includes a single "imp"-style export named `analysis_export`, with a pure virtual declaration of `write()`. To use this class, you must extend it and override the `write()` function to perform the analysis calculation.



```
class coverage_sb extends uvm_subscriber #(Packet);
  `uvm_component_utils(coverage_sb)

  Packet pkt;
  int pkt_cnt;

  covergroup cov1;
    s: coverpoint pkt.src_id {
      bins src[8] = {[0:7]};
    }
    d: coverpoint pkt.dest_id {
      bins dest[8] = {[0:7]};
    }
  }
```

```

    cross s, d;
endgroup : cov1

function new( string name , uvm_component parent);
    super.new( name , parent );
    cov1 = new();
endfunction

function void write(Packet t);
    real current_coverage;
    pkt = t;
    pkt_cnt++;
    cov1.sample(); // cause sampling of coverage
    current_coverage = $get_coverage();
    `uvm_info("COVERAGE",$sformatf("%0d Packets sampled, Coverage =
%f%% ",
                                pkt_cnt,current_coverage), UVM_MEDIUM)

endfunction

endclass

```

Two styles of implementation

There are two approaches you can take to implement the behavior of an analysis component with more than one input stream. The first approach using the ``uvm_analysis_imp_decl` macro is recommended as it provides better performance and uses less memory. The second approach using instances of `uvm_tlm_analysis_fifo` is a more traditional approach where a FIFO is created and testbench threads are spawned.

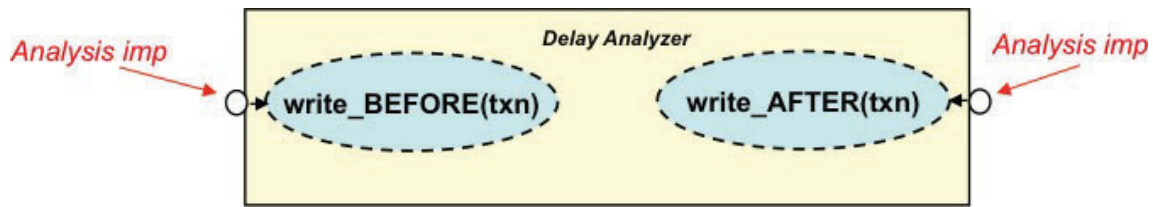
In one approach, you implement all the `write()` functions directly, and declare `imp` connections in the containing component. Because the `write()` functions must be non-blocking, you cannot directly perform any synchronization with other streams. If synchronization is required, you must deposit the transaction in some kind of storage element (e.g. a FIFO or associative array) and provide a separate process, usually in the `run()` task, to perform the synchronization.

In the other approach, you provide an embedded analysis FIFO for each input stream, and declare a hierarchical analysis export to expose the FIFO externally. In this case, the `write()` functions are provided by the FIFOs and they deposit the transactions into the FIFO data structures. You then implement the functionality of the analysis component through the `run()` task. Using this approach involves an extra step of connecting the analysis FIFOs to the analysis exports.

Multiple Transaction Streams Without Synchronization

When the analysis component needs to sample multiple transaction streams, it adds complications to writing the behavior of the component.

One complication when using `uvm_analysis_imps` with multiple streams is that each `uvm_analysis_imp` expects to be tied to a `write()` function. Since there can only be one function named `write()` in the component, a workaround is needed. UVM provides a workaround by defining the ``uvm_analysis_imp_decl` macro. This macro allows you to declare a specialized "imp"-style analysis export that calls a function named `write_SUFFIX` where you specify `_SUFFIX`. The ``uvm_analysis_imp_decl` macro is creating an extension of the `uvm_analysis_imp` class with the `_SUFFIX` used in the extension name that defines the `write_SUFFIX` function.



```
// Declare the suffixes that will be appended to the imps and functions
`uvm_analysis_imp_decl(_BEFORE)
`uvm_analysis_imp_decl(_AFTER)

class delay_analyzer extends uvm_component;
`uvm_component_utils(delay_analyzer)

// Declare the imps using the suffixes declared above.
// The first parameter is the transaction type.
// The second parameter is the type of component that implements the
interface
// Usually, the second parameter is the same as the containing
component type
    uvm_analysis_imp_BEFORE #(alu_txn, delay_analyzer) before_export;
    uvm_analysis_imp_AFTER      #(alu_txn, delay_analyzer) after_export;

    real m_before[$]; real
    m_after[$]; real
    last_b_time,
        last_a_time;
    real longest_b_delay,
        longest_a_delay;

    function new( string name , uvm_component parent) ;
        super.new( name , parent ); last_b_time
        = 0.0;
        last_a_time = 0.0;
    endfunction

// Implement the interface function by appending the function name with
// the suffix declared in the macro above.
    function void write_BEFORE(alu_txn t);
        real delay;
        delay = $realtime - last_b_time; last_b_time =
        $realtime; m_before.push_back(delay);
    endfunction

// Implement the interface function by appending the function name with
// the suffix declared in the macro above.
    function void write_AFTER(alu_txn t);
```



```

    real delay;
    delay = $realtime - last_a_time; last_a_time =
    $realtime; m_after.push_back(delay);
endfunction

function void build_phase( uvm_phase phase );
// The second argument to the imp constructor is a handle to the object
// that implements the interface functions. It should be of the type
// specified in the declaration of the imp. Usually, it is "this".
    before_export = new("before_export", this); after_export =
    new("after_export", this);
endfunction

function void extract_phase( uvm_phase phase );
    foreach (m_before[i])
        if (m_before[i] > longest_b_delay) longest_b_delay = m_before[i];
    foreach (m_after[i])
        if (m_after[i] > longest_a_delay) longest_a_delay = m_after[i];
endfunction

function void check_phase( uvm_phase phase );
    if (longest_a_delay > 100.0) begin
        `uvm_warning("Delay Analyzer", $sformatf("Transaction delay too long: %5.2f",
longest_a_delay))
    end
    if (longest_b_delay > 100.0) begin
        `uvm_warning("Delay Analyzer", $sformatf("Transaction delay too long: %5.2f",
longest_a_delay))
    end
endfunction

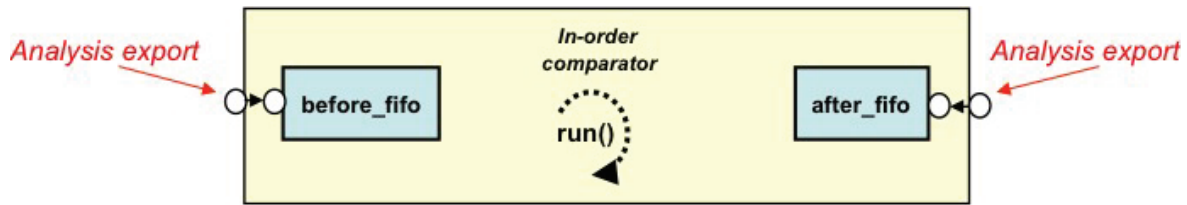
function void report_phase( uvm_phase phase );
    `uvm_info("Delay Analyzer", $sformatf("Longest BEFORE delay:
%5.2f", longest_b_delay), UVM_LOW)
    `uvm_info("Delay Analyzer", $sformatf("Longest AFTER delay:
%5.2f", longest_a_delay), UVM_LOW)
endfunction

endclass

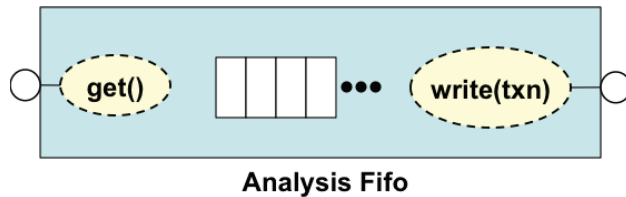
```

Multiple Transaction Streams With Synchronization

Implementing the write() functions directly is easier when the required behavior of the analysis component allows for each stream's processing to be done independently. Many situations, however, require synchronization between the transaction streams, so that data is not lost. For example, an in-order Comparator has two streams, one from the Predictor and one from the Monitor. It must wait until it has a matching pair of transactions, one from each stream, to compare.



In this case, rather than performing all the synchronization work manually in two write() functions, you can instantiate two analysis FIFOs and place two hierarchical exports on the Comparator. Since you are not implementing the behavior of the Comparator in the write() functions, a different approach is required. In this case, you can implement the Comparator behavior in the run() task of the component and use the blocking operations on the analysis FIFOs to perform the necessary synchronization.



```
class comparator_inorder extends uvm_component;
`uvm_component_utils(comparator_inorder)

uvm_analysis_export #(alu_txn) before_export;
uvm_analysis_export #(alu_txn) after_export;

uvm_tlm_analysis_fifo #(alu_txn) before_fifo,
                                after_fifo;

int m_matches,
    m_mismatches;

function new( string name , uvm_component parent ) ;
    super.new( name , parent );
    m_matches = 0;
    m_mismatches = 0;
endfunction

function void build_phase( uvm_phase phase );
    before_fifo = new("before_fifo", this);
    after_fifo = new("after_fifo", this);
    before_export = new("before_export", this);
    after_export = new("after_export", this);
endfunction

function void connect_phase( uvm_phase phase );
    before_export.connect(before_fifo.analysis_export);
    after_export.connect(after_fifo.analysis_export);
endfunction

task run_phase( uvm_phase phase );
```

```
alu_txn before_txn, after_txn;
forever begin
  before_fifo.get(before_txn);
  after_fifo.get(after_txn);
  if (!before_txn.compare(after_txn)) begin
    `uvm_error("Comparator Mismatch",
              $sformatf("%s does not match %s",
before_txn.convert2string(), after_txn.convert2string()))
    m_mismatches++;
  end else begin
    m_matches++;
  end
end
endtask

function void report_phase( uvm_phase phase );
  `uvm_info("Inorder Comparator", $sformatf("Matches:    %0d",
m_matches), UVM_LOW)
  `uvm_info("Inorder Comparator", $sformatf("Mismatches: %0d",
m_mismatches), UVM_LOW)
endfunction

endclass
```

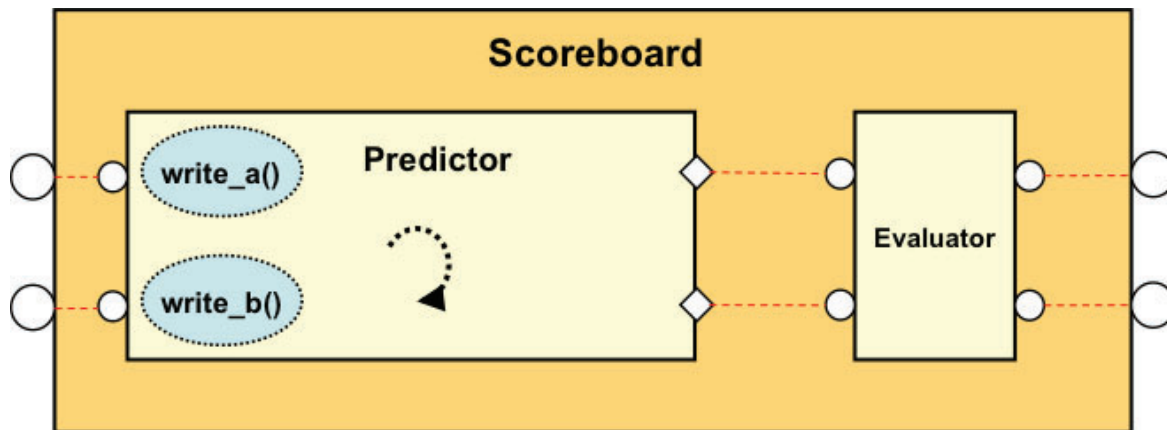
For more complicated synchronization needs, you would use a combination of multiple `write_SUFFIX()` functions which would place transaction data into some kind of shared data structure, along with code in the `run()` task to perform coordination and control.

Predictors

Overview

A Predictor is a verification component that represents a "golden" reference model of all or part of the DUT functionality. It takes the same input stimulus that is sent to the DUT and produces expected response data that is by definition correct. When you send random stimulus into your DUT, you need an automatic way to check the result as you can no longer check the output by hand. A Predictor generates expected output that is compared to the actual DUT output to give a pass / fail.

Predictors in the Testbench Environment



Predictors are the part of the Scoreboard component that generates expected transactions. They should be separate from the part of the Scoreboard that performs the evaluation. A Predictor can have one or more input streams, which are the same input streams that are applied to the DUT.

Construction

Predictors are typical analysis components that are subscribers to transaction streams. The inputs to a Predictor are transactions generated from monitors observing the input interfaces of the DUT. The Predictors take the input transaction(s) and process them to produce expected output transactions. Those output transactions are broadcast through analysis ports to the evaluator part of the scoreboard, and to any other analysis component that needs to observe predicted transactions. Internally, Predictors can be written in C, C++, SV or SystemC, and are written at an abstract level of modeling. Since Predictors are written at the transaction level, they can be readily chained if needed.

Example

```
class alu_tlm extends uvm_subscriber #(alu_txn);
  `uvm_component_utils(alu_tlm)

  uvm_analysis_port #(alu_txn) results_ap;

  function new(string name, uvm_component parent );
    super.new( name , parent );
  endfunction

  function void build_phase( uvm_phase phase );
```

```
results_ap = new("results_ap", this);
endfunction

function void write( alu_txn t);
alu_txn out_txn;
$cast(out_txn,t.clone());
case(t.mode)
  ADD: out_txn.result = t.val1 + t.val2;
  SUB: out_txn.result = t.val1 - t.val2;
  MUL: out_txn.result = t.val1 * t.val2;
  DIV: out_txn.result = t.val1 / t.val2;
endcase
results_ap.write(out_txn);
endfunction

endclass
```

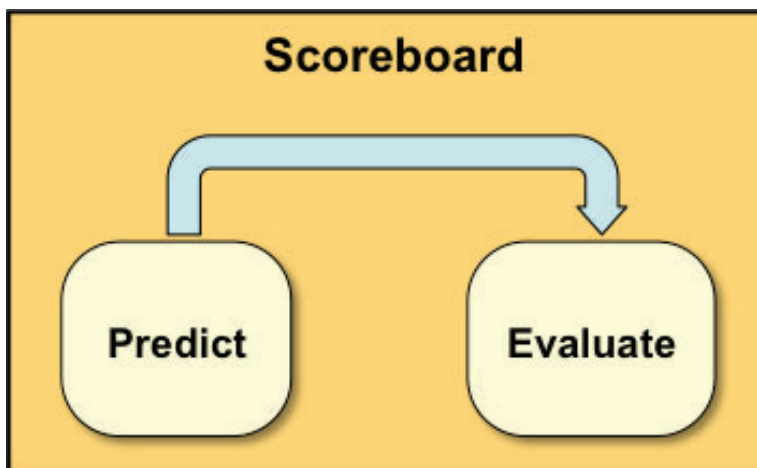
Predictor as a Proxy for the DUT

Another use of a Predictor is to act as a proxy DUT while the DUT is being written. Typically, since the Predictor is written at a higher level of abstraction, it takes less time to write and is available earlier than the DUT. As a proxy for the DUT, it allows testbench development and debugging to proceed in parallel with DUT development.

Scoreboards

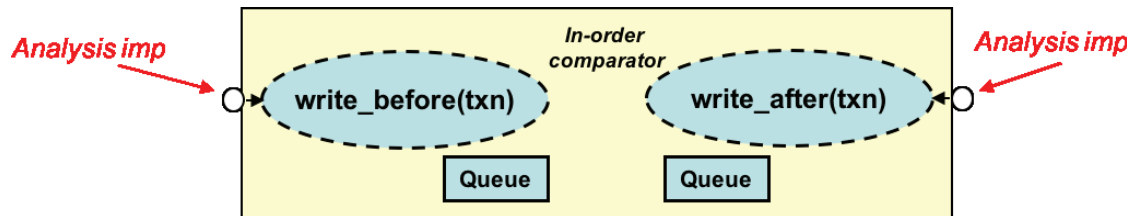
Overview

The Scoreboard's job is to determine whether or not the DUT is functioning properly. The scoreboard is usually the most difficult part of the testbench to write, but it can be generalized into two parts: The first step is determining what exactly is the correct functionality. Once the correct functionality is predicted, the scoreboard can then evaluate whether or not the actual results observed on the DUT match the predicted results. The best scoreboard architecture is to separate the prediction task from the evaluation task. This gives you the most flexibility for reuse by allowing for substitution of predictor and evaluation models, and follows the best practice of separation of concerns.



In cases where there is a single stream of predicted transactions and a single stream of actual transactions, the scoreboard can perform the evaluation with a simple comparator. The most common comparators are an in-order and out-of-order comparator.

Comparing Transactions Assuming In-Order Arrival



An in-order comparator assumes that matching transactions will appear in the same order from both expected and actual streams. It gets transactions from the expected and actual side and evaluates them. The transactions will arrive independently, but in order so transactions can be stored from the "before" side and then compared when a transaction arrives on the "after" side. Evaluation can be as simple as calling the transaction's `compare()` method, or it can be more involved, because for the purposes of evaluating correct behavior, comparison does not necessarily mean equality.

```
`uvm_analysis_imp_decl(_before)
`uvm_analysis_imp_decl(_after)

class comparator_inorder extends uvm_component;

    `uvm_component_utils(comparator_inorder)

    uvm_analysis_imp_before #(T, comparator_inorder) before_export;
    uvm_analysis_imp_after #(T, comparator_inorder) after_export;

    int m_matches, m_mismatches;
    protected T m_before[$];
    protected T m_after[$];

    function new( string name , uvm_component parent ) ;
        super.new( name , parent );
        m_matches = 0;
        m_mismatches = 0;
    endfunction

    function void build_phase( uvm_phase phase );
        before_export = new("before_export", this);
        after_export = new("after_export", this);
    endfunction

    protected virtual function void m_proc_data();
        T bef = m_before.pop_front();
        T aft = m_after.pop_front();
        if (!bef.compare(aft)) begin
            `uvm_error("Comparator Mismatch",
                $sformatf("%s does not match %s",
                    bef.convert2string(),
                    aft.convert2string()))
            m_mismatches++;
        end
    end
endclass
```

```

end
else begin
    m_matches++;
end
endfunction

virtual function void write_before(T txn);
    m_before.push_back(txn);
    if (m_after.size())
        m_proc_data();
endfunction

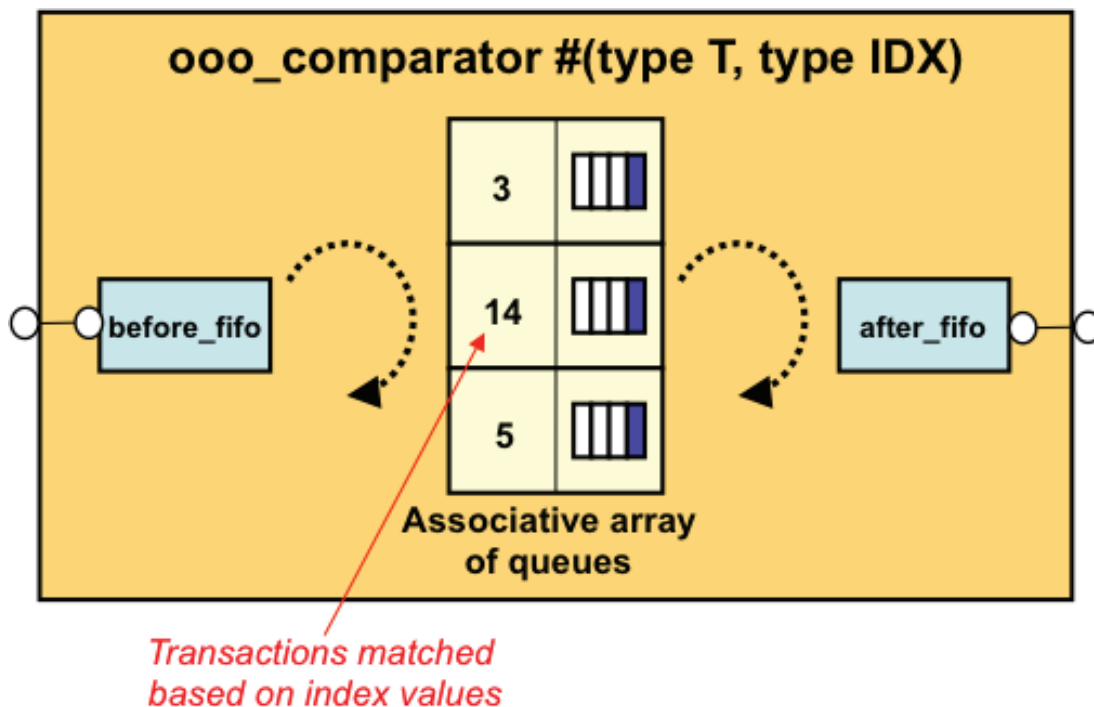
virtual function void write_after(T txn);
    m_after.push_back(txn);
    if (m_before.size())
        m_proc_data();
endfunction

function void report_phase( uvm_phase phase );
    `uvm_info("Inorder Comparator", $sformatf("Matches:    %0d",
m_matches), UVM_LOW);
    `uvm_info("Inorder Comparator", $sformatf("Mismatches: %0d",
m_mismatches), UVM_LOW);
endfunction

endclass

```

Comparing transactions out-of-order



An out-of-order comparator makes no assumption that matching transactions will appear in the same order from the expected and actual sides. So, unmatched transactions need to be stored until a matching transaction appears on the opposite stream. For most out-of-order comparators, an associative array is used for storage. This example comparator has two input streams arriving through analysis exports. The implementation of the comparator is symmetrical, so the export names do not have any real importance. This example uses embedded fifos to implement the analysis write() functions, but since the transactions are either stored into the associative array or evaluated upon arrival, this example could easily be written using analysis imps and write() functions.

Because of the need to determine if two transactions are a match and should be compared, this example requires transactions to implement an index_id() function that returns a value that is used as a key for the associative array. If an entry with this key already exists in the associative array, it means that a transaction previously arrived from the other stream, and the transactions are compared. If no key exists, then this transaction is added to associative array.

This example has an additional feature in that it does not assume that the index_id() values are always unique on a given stream. In the case where multiple outstanding transactions from the same stream have the same index value, they are stored in a queue, and the queue is the value portion of the associative array. When matches from the other stream arrive, they are compared in FIFO order.

```
class ooo_comparator
#(type T = int,
  type IDX = int)
  extends uvm_component;

typedef ooo_comparator #(T, IDX) this_type;
`uvm_component_param_utils(this_type)

typedef T q_of_T[$];
typedef IDX q_of_IDX[$];

uvm_analysis_export #(T) before_axp, after_axp;

protected uvm_tlm_analysis_fifo #(T) before_fifo, after_fifo;
bit before_queued = 0;
bit after_queued = 0;

protected int m_matches, m_mismatches;

protected q_of_T received_data[IDX];
protected int rcv_count[IDX];

protected process before_proc = null;
protected process after_proc = null;

function new(string name, uvm_component parent);
  super.new(name, parent);
endfunction

function void build_phase( uvm_phase phase );
  before_axp = new("before_axp", this);
```



```

    after_axp = new("after_axp", this); before_fifo =
    new("before", this); after_fifo = new("after", this);
endfunction

function void connect_phase( uvm_phase phase );
    before_axp.connect(before_fifo.analysis_export);
    after_axp.connect(after_fifo.analysis_export);
endfunction : connect

// The component forks two concurrent instantiations of this task
// Each instantiation monitors an input analysis fifo
protected task get_data(ref uvm_tlm_analysis_fifo #(T) txn_fifo,
input bit is_before);
    T txn_data, txn_existing; IDX idx;
    string rs; q_of_T
    tmpq;
    bit need_to_compare;
    forever begin

        // Get the transaction object, block if no transaction available
        txn_fifo.get(txn_data); idx =
        txn_data.index_id();

        // Check to see if there is an existing object to compare
        need_to_compare = (rcv_count.exists(idx) &&
                           ((is_before && rcv_count[idx] > 0) || (!is_before &&
                           rcv_count[idx] < 0)));

        if (need_to_compare) begin
            // Compare objects using compare() method of transaction
            tmpq = received_data[idx]; txn_existing =
            tmpq.pop_front(); received_data[idx] = tmpq;
            if (txn_data.compare(txn_existing)) m_matches++;
        else
            m_mismatches++;
        end
    else begin
        // If no compare happened, add the new entry
        if (received_data.exists(idx)) tmpq =
        received_data[idx];
    else
        tmpq = {};
        tmpq.push_back(txn_data);
        received_data[idx] = tmpq;
    end
end

```

```

    end

    // Update the index count
    if (is_before)
        if (rcv_count.exists(idx)) begin
            rcv_count[idx]--;
        end
        else
            rcv_count[idx] = -1;
        else
            if (rcv_count.exists(idx)) begin
                rcv_count[idx]++;
            end
            else
                rcv_count[idx] = 1;
            end
        end

    // If index count is balanced, remove entry from the arrays
    if (rcv_count[idx] == 0) begin
        received_data.delete(idx);
        rcv_count.delete(idx);
    end
end // forever
endtask

virtual function int get_matches();
    return m_matches;
endfunction : get_matches

virtual function int get_mismatches();
    return m_mismatches;
endfunction : get_mismatches

virtual function int get_total_missing();
    int num_missing;
    foreach (rcv_count[i]) begin
        num_missing += (rcv_count[i] < 0 ? -rcv_count[i] : rcv_count[i]);
    end
    return num_missing;
endfunction : get_total_missing

virtual function q_of_IDX get_missing_indexes(); q_of_IDX rv =
    rcv_count.find_index() with (item != 0); return rv;
endfunction : get_missing_indexes;

virtual function int get_missing_index_count(IDX i);
    // If count is < 0, more "before" txns were received

```

```
// If count is > 0, more "after" txns were received
    if (rcv_count.exists(i))
        return rcv_count[i];
    else
        return 0;
endfunction : get_missing_index_count;

task run_phase( uvm_phase phase );
    fork
        get_data(before_fifo, before_proc, 1);
        get_data(after_fifo, after_proc, 0);
    join
endtask : run_phase

endclass : ooo_comparator
```

Advanced Scenarios

In more advanced scenarios, there can be multiple predicted and actual transaction streams coming from multiple DUT interfaces. In this case, a simple comparator is insufficient and the implementation of the evaluation portion of the scoreboard is more complex and DUT-specific.

Reporting and Recording

The result of the evaluation is a boolean value, which the Scoreboard should use to report and record failures. Usually successful evaluations are not individually reported, but can be recorded for later summary reports.

Metric Analyzers

Overview

Metric Analyzers watch and record non-functional behavior such as latency, power utilization, and other performance-related measurements.

Construction

Metric Analyzers are generally standard analysis components. They implement their behavior in a way that depends on the number of transaction streams they observe - either by extending `uvm_subscriber` or with analysis imp/exports. They can perform ongoing calculations during the run phase, and/or during the post-run phases.

Example

```
`uvm_analysis_imp_decl(_BEFORE)
`uvm_analysis_imp_decl(_AFTER)

class delay_analyzer extends uvm_component;
`uvm_component_utils(delay_analyzer)

    uvm_analysis_imp_BEFORE #(alu_txn, delay_analyzer) before_export;
    uvm_analysis_imp_AFTER  #(alu_txn, delay_analyzer) after_export;

    real m_before[$];
    real m_after[$];
    real last_b_time, last_a_time;
    real longest_b_delay, longest_a_delay;

    function new( string name , uvm_component parent) ;
        super.new( name , parent );
        last_b_time = 0.0;
        last_a_time = 0.0;
    endfunction

    // Record when the transaction arrives
    function void write_BEFORE(alu_txn t);
        real delay;
        delay = $realtime - last_b_time;
        last_b_time = $realtime;
        m_before.push_back(delay);
    endfunction

    // Record when the transaction arrives
    function void write_AFTER(alu_txn t);
        real delay;
        delay = $realtime - last_a_time;
        last_a_time = $realtime;
        m_after.push_back(delay);
```

```
endfunction

function void build_phase( uvm_phase phase );
    before_export = new("before_export", this);
    after_export = new("after_export", this);
endfunction

// Perform calculation for longest delay metric
function void extract_phase( uvm_phase phase );
    foreach (m_before[i])
        if (m_before[i] > longest_b_delay) longest_b_delay = m_before[i];

    foreach (m_after[i])
        if (m_after[i] > longest_a_delay) longest_a_delay = m_after[i];
endfunction

function void check_phase( uvm_phase phase );
    string s;
    if (longest_a_delay > 100.0) begin
        $sformat(s, "Transaction delay too long: %5.2f", longest_a_delay);
        `uvm_warning("Delay Analyzer", s);
    end
    if (longest_b_delay > 100.0) begin
        $sformat(s, "Transaction delay too long: %5.2f", longest_b_delay);
        `uvm_warning("Delay Analyzer", s);
    end
endfunction

function void report_phase( uvm_phase phase );
    `uvm_info("Delay Analyzer", $sformatf("Longest BEFORE delay:
%5.2f", longest_b_delay), UVM_LOW);
    `uvm_info("Delay Analyzer", $sformatf("Longest AFTER delay:
%5.2f", longest_a_delay), UVM_LOW);
endfunction

endclass
```

Post-Run Phases

Overview

Many analysis components perform their analysis on an ongoing basis during the simulation run. Sometimes you need to defer analysis until all data has been collected, or a component might need to do a final check at the end of simulation. For these components, UVM provides the post-run phases extract, check, and report.

These phases are executed in a hierarchically bottom-up fashion on all components.

The Extract Phase

The extract phase allows a component to examine data collected during the simulation run, extract meaningful values and perform arithmetic computation on those values.

The Check Phase

The check phase allows a component to evaluate any values computed during the extract phase and make a judgment about whether the values are correct. Also, for components that perform analysis continuously during the run, the check phase can be used to check for any missing data or extra data such as unmatched transactions in a scoreboard.

The Report Phase

The report phase allows a component to display a final report about the analysis in its area of responsibility. A component can be configured whether or not to display its local results, to allow for accumulation and display by higher-level components.

The Final Phase

The Final phase is the very last UVM phase to be executed before the UVM executes \$finish to bring the simulation to an end.

Example

```
`uvm_analysis_imp_decl(_BEFORE)
`uvm_analysis_imp_decl(_AFTER)

class delay_analyzer extends uvm_component;
`uvm_component_utils(delay_analyzer)

    uvm_analysis_imp_BEFORE #(alu_txn, delay_analyzer) before_export;
    uvm_analysis_imp_AFTER  #(alu_txn, delay_analyzer) after_export;

    real m_before[$];
    real m_after[$];
    real last_b_time, last_a_time;
    real longest_b_delay, longest_a_delay;

    function new( string name , uvm_component parent) ;
        super.new( name , parent );
        last_b_time = 0.0;
        last_a_time = 0.0;
```

```

endfunction

function void write_BEFORE(alu_txn t);
    real delay;
    delay = $realtime - last_b_time; last_b_time =
    $realtime; m_before.push_back(delay);
endfunction

function void write_AFTER(alu_txn t);
    real delay;
    delay = $realtime - last_a_time; last_a_time =
    $realtime; m_after.push_back(delay);
endfunction

function void build_phase( uvm_phase phase ); before_export =
    new("before_export", this); after_export = new("after_export",
    this);
endfunction

function void extract_phase( uvm_phase phase );
    foreach (m_before[i])
        if (m_before[i] > longest_b_delay) longest_b_delay = m_before[i];

    foreach (m_after[i])
        if (m_after[i] > longest_a_delay) longest_a_delay = m_after[i];
endfunction

function void check_phase( uvm_phase phase );
    string s;
    if (longest_a_delay > 100.0) begin
        $sformat(s, "Transaction delay too long: %5.2f",longest_a_delay);
        `uvm_warning("Delay Analyzer",s);
    end
    if (longest_b_delay > 100.0) begin
        $sformat(s, "Transaction delay too long: %5.2f",longest_a_delay);
        `uvm_warning("Delay Analyzer",s);
    end
endfunction

function void report_phase( uvm_phase phase );
    `uvm_info("Delay Analyzer", $sformatf("Longest BEFORE delay:
    %5.2f", longest_b_delay), UVM_LOW);
    `uvm_info("Delay Analyzer", $sformatf("Longest AFTER delay:
    %5.2f", longest_a_delay), UVM_LOW);

```

```
endfunction

function void final_phase( uvm_phase phase );
    my_summarize_test_results();
endfunction

endclass
```